

Study Guide for Greedy Algorithms

This study guide provides a comprehensive review of greedy algorithms, covering fundamental concepts, classic problems, and advanced theoretical frameworks. It includes a short-answer quiz with an answer key, a set of essay questions for deeper analysis, and a complete glossary of key terms as defined in the source materials.

Short-Answer Quiz

Instructions: Answer the following questions in 2-3 sentences each, based on the provided source materials.

1. What are the two fundamental properties an optimization problem must exhibit for a correct greedy algorithm to be applicable?
2. Explain why the standard greedy coin-changing algorithm is not optimal for giving 6 cents of change with denominations $\{4, 3, 1\}$.
3. What is the optimal greedy strategy for the Interval Scheduling problem, and what is the intuition behind its effectiveness?
4. Briefly describe the main steps of Huffman's algorithm for constructing an optimal prefix code.
5. What is a matroid, and why is this concept important for greedy algorithms?
6. What is a prefix-free code, and why is this property essential for decoding messages?
7. What is the difference in the objective between the Interval Scheduling problem and the Interval Partitioning problem?
8. In the context of the Interval Partitioning problem, what is the "depth" of a set of intervals, and what is its significance?
9. How does a variable-length code potentially offer better compression than a fixed-length code?
10. What is an M-augmenting path in the context of bipartite graph matching?

Answer Key

1. A problem must exhibit **optimal substructure**, meaning an optimal solution to the problem contains optimal solutions to its subproblems. It must also have the **greedy-choice property**, which means a globally optimal solution can be reached by making a series of locally optimal (greedy) choices.
2. The greedy algorithm would first select the largest coin that fits, which is a 4-cent coin, leaving 2 cents. It would then select two 1-cent coins, for a total of three coins ($4+1+1$). This is not optimal because a solution with two 3-cent coins exists, which uses only two coins.
3. The optimal strategy is "earliest finish time first," which considers jobs in ascending order of their finishing times. The intuition is that this strategy frees up the shared

resource as early as possible, thereby maximizing the opportunity for more compatible jobs to be scheduled later.

4. The algorithm repeatedly identifies the two characters with the smallest frequencies from a priority queue and merges them into a new subtree. The parent node of this subtree is assigned a frequency equal to the sum of its children's frequencies and is inserted back into the queue. This merge process continues until only one node, the root of the complete Huffman tree, remains.
5. A matroid is a structure consisting of a finite set and a nonempty family of its subsets that is hereditary and satisfies the exchange property. Matroids are important because they characterize a class of problems for which the generic greedy algorithm is guaranteed to yield an optimal solution.
6. A prefix-free code is one where no code word is a prefix of any other code word. This property is essential because it ensures that any encoded message is uniquely decodable; a decoder can read bits until a valid code word is matched, peel it off, and continue decoding without ambiguity.
7. In Interval Scheduling, the goal is to find the maximum subset of mutually compatible jobs that can run on a single resource. In contrast, the goal of Interval Partitioning is to schedule *all* given jobs using the minimum number of resources (e.g., classrooms) required.
8. The depth is the maximum number of intervals that contain any single given point in time. Its significance is that the number of classrooms needed for any schedule must be at least the depth, and the "earliest start time first" greedy algorithm is proven to find a schedule that uses a number of classrooms exactly equal to the depth.
9. A variable-length code can assign shorter code words to more frequent characters and longer code words to less frequent characters. This reduces the average number of bits needed per character compared to a fixed-length code, where every character uses the same number of bits regardless of its frequency.
10. Given a matching M in a bipartite graph, an M -augmenting path is a path whose edges alternate between being unmatched (not in M) and matched (in M). The endpoints of this path must be free, or unmatched, vertices.

Essay Questions

Instructions: The following questions are designed for longer-form, detailed answers. Construct your arguments and explanations using concepts and examples from the source materials.

1. Compare and contrast the optimal greedy strategies for the Interval Scheduling problem and the Interval Partitioning problem. Explain why "earliest finish time first" is optimal for the former, while "earliest start time first" is optimal for the latter.
2. Walk through the proof of optimality for the Huffman coding algorithm. Explain the role of the key lemma (regarding the two letters with the smallest frequency being sibling leaves) and the use of induction in the main proof.
3. Using the coin-changing problem as a primary example, discuss the conditions under which a greedy algorithm provides an optimal solution versus when it provides a non-

optimal heuristic. Use the specific coin denominations $\{5, 2, 1\}$ and $\{4, 3, 1\}$ to illustrate both cases and their underlying properties.

4. Define a matroid using its two formal properties (hereditary and exchange). Explain how Kruskal's algorithm for finding a minimum spanning tree can be understood as a generic greedy algorithm operating on a specific type of matroid, the graphic matroid.
5. Explain the concepts of optimal substructure and the greedy-choice property in detail. How do these two properties work together to allow for a proof of correctness for a greedy algorithm? Provide an example of a problem discussed in the sources that exhibits both properties and explain how they apply.

Glossary

Term	Definition
Augmenting Path (M-augmenting path)	A path in a bipartite graph, relative to a matching M , where the edges alternate between being free (unmatched) and matched. The endpoints of the path are free vertices.
Binary Code	An encoding that represents each character in an alphabet as a binary string, also known as a code word.
Bipartite Matching	A set of edges in a bipartite graph where no two edges share a common vertex.
Depth (of a set of open intervals)	The maximum number of intervals that contain a given point in time.
Exchange Property	A property of a matroid (S, F) . It states that if A and B are in F and
Fixed-Length Code	A binary code where each character's code word has the same length.
Greedy Algorithm	An algorithm that solves an optimization problem by working in phases. In each phase, it makes a decision that is locally optimal given the information obtained so far, without regard for future consequences.
Greedy-Choice Property	The property that a globally optimal solution can be arrived at by making a series of locally optimal (greedy) choices.
Graphic Matroid	A matroid defined on an undirected graph $G=(V, E)$, where S is the set of edges E , and F is the family of all subsets of edges A such that the subgraph (V, A) is a forest.
Hereditary Property	A property of a matroid (S, F) . It states that if a set B is in the family F , then any subset A of B must also be in F .
Huffman Code	An optimal (minimum cost) prefix-free code generated by the Huffman algorithm.
Interval Partitioning	An optimization problem where the goal is to schedule all given lectures (or jobs) using the minimum number of classrooms (or resources) so that no two lectures occur at the same time in the same room.
Interval Scheduling	An optimization problem where the goal is to find a maximum subset of mutually compatible jobs from a set of jobs, each with a start and finish time, that can be run on a single shared resource.

Kruskal's Algorithm	A greedy algorithm for finding a minimum spanning tree in a weighted, connected graph. It considers edges in increasing order of weight and adds an edge if and only if it does not form a cycle.
Marriage Theorem	A theorem for a bipartite graph $G = (W + X, E)$ which states that all vertices in W can be matched ($m(G) =$
Matric Matroid	A matroid defined on a matrix M , where S is the set of rows of M , and F is the family of all subsets of rows that are linearly independent.
Matroid	A structure (S,F) where S is a finite set and F is a nonempty family of subsets of S that is hereditary and satisfies the exchange property. They characterize problems for which a greedy algorithm is optimal.
Minimum Spanning Tree (MST)	For a connected, weighted graph, an MST is a spanning tree with the minimum possible total edge weight.
Optimal Substructure	A property of an optimization problem where an optimal solution to the problem contains within it optimal solutions to subproblems.
Prefix-Free Code	A type of binary code where no code word is a prefix of any other code word. This property guarantees that any message encoded with it is uniquely decodable.
Spanning Tree	For a connected graph G , a spanning tree is an induced subgraph that is a tree and connects all vertices of G .
Transversal	In a bipartite graph $G = (W + X, E)$, a subset of W that can be matched is called a partial transversal.
Transversal Matroid	A matroid defined on a bipartite graph $G = (W + X, E)$, where the set is W and the family of subsets is the collection of all transversals of G .
Uniquely Decodable	A property of a code where any encoded message can be decoded in only one way, returning the original message.
Variable-Length Code	A binary code where code words for different characters may have different lengths.
Weighted External Path Length	For a binary tree representing a code, it is the sum over all leaves of the frequency of the character at that leaf multiplied by the depth of the leaf. Minimizing this value is equivalent to finding an optimal code.
Weighted Matroid	A matroid equipped with a weight function that assigns a non-negative real number to each element in its underlying set S .